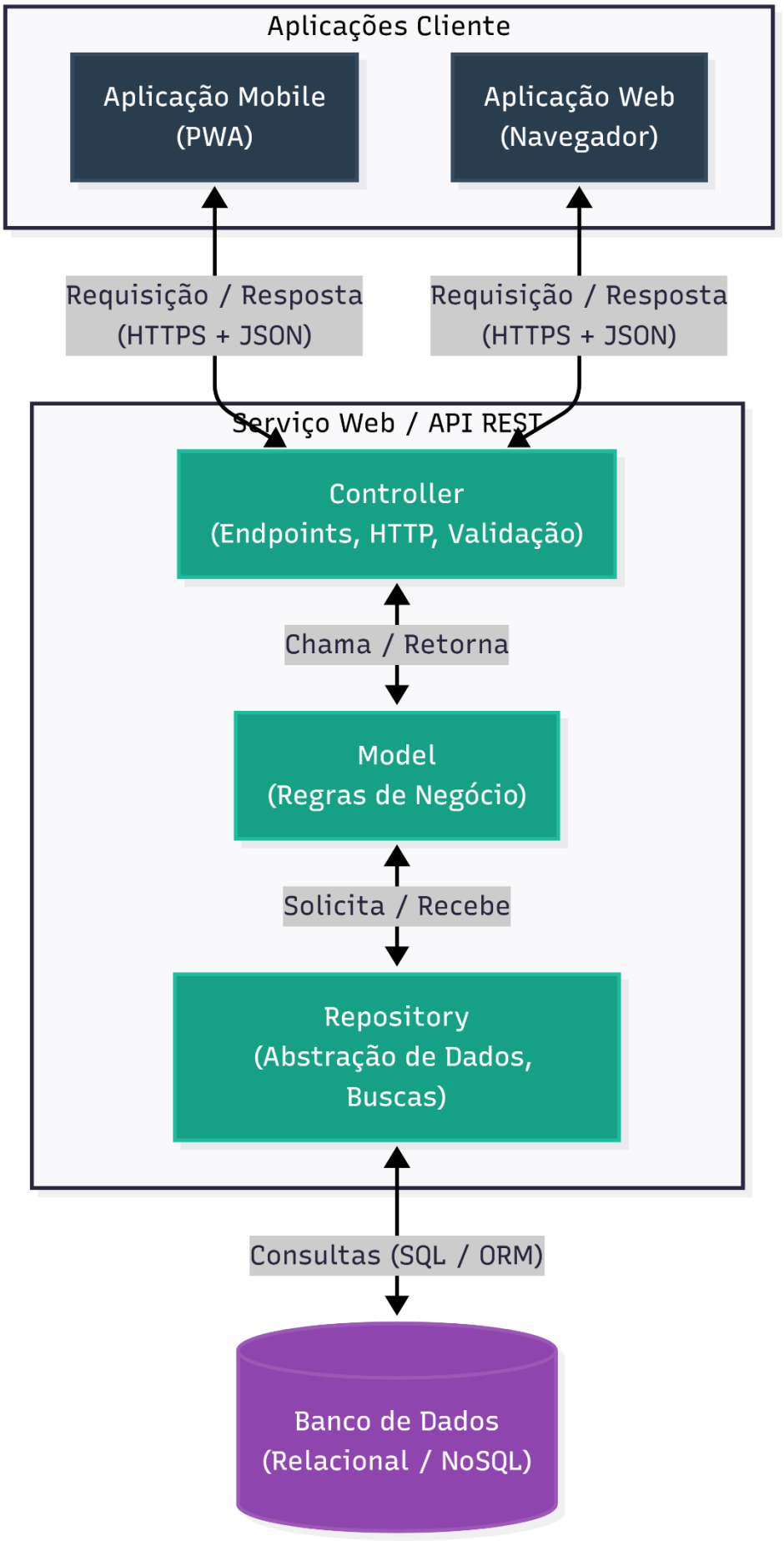
Aplicações Cliente
Aplicação Mobile (PWA)
Aplicação Web (Navegador)
Requisição / Resposta (HTTPS + JSON)
Requisição / Resposta (HTTPS + JSON)
Serviço Web / API REST
Controller (Endpoints, HTTP, Validação)
Chama / Retorna
Model (Regras de Negócio)
Solicita / Recebe
Repository (Abstração de Dados, Buscas)
Consultas (SQL / ORM)
Banco de Dados (Relacional / NoSQL)

1. O Papel do Serviço Web (Backend)

Na arquitetura proposta, o **Serviço Web atua como o núcleo central (Backend-as-a-Service ou API RESTful)** do sistema. Ele centraliza toda a inteligência da aplicação, garantindo que o Web e o Mobile funcionem apenas como interfaces de usuário (Frontends).

Dentro do padrão arquitetural de software (como o MVC adaptado para APIs):

- **Controller (Controlador):** É a porta de entrada do Serviço Web. Ele recebe as requisições (requests) vindas das aplicações Web e Mobile. Seu papel é expor os *endpoints* (URLs da API), interceptar a chamada, extrair os dados enviados pelos clientes (parâmetros, corpo da requisição), validar se a requisição está bem formatada e encaminhar a ação para a camada de Model. Ao final, ele recebe a resposta do Model e a formata para devolver ao cliente com o código de status HTTP adequado (ex: 200 OK, 201 Created, 400 Bad Request).
- **Model (Regras de Negócio e Acesso a Dados):** É o coração do sistema. O Model garante que as lógicas e restrições da empresa sejam aplicadas de forma unificada. Independentemente de a requisição vir do celular ou do navegador, a regra para "calcular um desconto" ou "validar o saldo" é executada no mesmo lugar.
    - *Acesso a Dados:* Após processar a regra de negócio, a camada de Model (frequentemente usando um ORM - *Object-Relational Mapper* ou repositórios) interage com o Banco de Dados para realizar operações de CRUD (Criar, Ler, Atualizar, Deletar). O banco de dados é blindado; as aplicações clientes nunca o acessam diretamente, apenas através do Serviço Web.

---

2. Consumo pelas Aplicações Web e Mobile

Tanto a Aplicação Web (ex: desenvolvida em React, Angular ou Vue.js) quanto a Aplicação Mobile (ex: React Native, Flutter, Swift ou Kotlin) atuarão como **clientes (consumers)** do Serviço Web.

- **Desacoplamento:** Nenhuma regra de negócio crítica ou conexão direta ao banco de dados reside no Web ou no Mobile. Eles são responsáveis apenas pela **View** (apresentação visual e interação com o usuário).
- **Ação:** Quando o usuário clica em um botão "Salvar" no aplicativo Mobile ou no site, o cliente empacota os dados preenchidos na tela e faz uma chamada assíncrona (via bibliotecas como fetch, axios ou Retrofit) para a URL do Serviço Web correspondente. O cliente então aguarda a resposta e atualiza a interface (exibindo uma mensagem de sucesso ou erro) com base no que o backend retornou.

---

3. Protocolos e Formatos de Dados Escolhidos

Protocolo: HTTP / HTTPS sob o padrão REST

A comunicação entre os clientes (Web/Mobile) e o Serviço Web será feita utilizando o protocolo **HTTPS (Hyper Text Transfer Protocol Secure)**, aplicando a arquitetura **REST (Representational State Transfer)**.

- **Justificativa Técnica:** O HTTP é o padrão universal da web. Utilizar REST permite mapear operações diretamente aos verbos semânticos do HTTP (GET para leitura, POST para criação, PUT/PATCH para atualização, DELETE para exclusão). O uso do "S" (Secure/TLS) é obrigatório para criptografar os dados em trânsito, protegendo informações sensíveis contra interceptações (*man-in-the-middle*). Além disso, APIs REST são *stateless* (sem estado), o que significa que o servidor não precisa guardar a sessão do usuário entre as chamadas, facilitando imensamente a escalabilidade (adicionar mais servidores se o tráfego aumentar).

Formato de Representação: JSON (JavaScript Object Notation)

O formato escolhido para envio e recebimento de dados (payload) entre os clientes e o servidor será o **JSON**.

- **Justificativa Técnica:**
- 1. **Leveza e Desempenho:** JSON consome muito menos largura de banda e exige menos poder de processamento para ser analisado em comparação com o XML. Isso é crucial para a aplicação Mobile, que pode estar operando em redes 3G/4G instáveis.
- 2. **Integração Nativa com a Web:** Como a aplicação Web será consumida por navegadores, o JSON possui compatibilidade nativa, tornando a serialização e desserialização triviais.
- 3. **Suporte Mobile:** Todas as linguagens modernas de desenvolvimento mobile possuem bibliotecas altamente otimizadas e maduras para lidar com JSON, tornando a integração de dados direta e tipada de maneira muito mais limpa.

2. Endpoints da API

A seguir, são apresentados os 10 endpoints essenciais para a operação do sistema Kourts, com suas respectivas descrições e exemplos de payloads.

2.1. Autenticação de Usuário

Este endpoint é responsável por autenticar um usuário no sistema e fornecer um token de acesso para requisições subsequentes.

- Recurso: Autenticação
- Endpoint: /api/auth/login
- Verbo HTTP: POST
- Ação: Valida as credenciais fornecidas (email/username e senha). Em caso de sucesso, gera e retorna um token de autenticação (ex: JWT) e informações adicionais.

Payload (Entrada):

{

"email": "usuario@example.com",

"senha": "minhasenha123" }

Payload (Saída - Sucesso):

{

"token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG9lIiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c",

"tipo": "Bearer",

"expiraEm": "2026-03-16T10:00:00Z" }

Payload (Saída - Erro 401 Unauthorized):

{

"status": 401,

"erro": "Unauthorized",

"mensagem": "Credenciais inválidas." }

2.2. Cadastro de Novo Usuário

Permite o registro de novos usuários no sistema.

- Recurso: Usuários
- Endpoint: /api/usuarios
- Verbo HTTP: POST
- Ação: Cadastra um novo usuário com os dados fornecidos no corpo da requisição.

Payload (Entrada):

{

"nome": "João Silva",

"email": "joao.silva@example.com",

"senha": "senhaSegura123",

"telefone": "5511987654321",

"dataNascimento": "1990-05-15" }

Payload (Saída - Sucesso 201 Created):

{

"id": 1,

"nome": "João Silva",

"email": "joao.silva@example.com",

"telefone": "5511987654321",

"dataNascimento": "1990-05-15",

"dataCadastro": "2026-03-16T09:30:00Z" }

Payload (Saída - Erro 400 Bad Request):

{

"status": 400,

"erro": "Bad Request",

"mensagem": "Email já cadastrado ou dados inválidos." }

2.3. Busca de Perfil de Usuário

Permite a recuperação do perfil de um usuário específico.

- Recurso: Usuários
- Endpoint: /api/usuarios/{id}
- Verbo HTTP: GET
- Ação: Busca e retorna os detalhes do perfil de um usuário pelo seu identificador único.

Payload (Entrada): N/A (ID fornecido na URL)

Payload (Saída - Sucesso 200 OK):

{

"id": 1,

"nome": "João Silva",

"email": "joao.silva@example.com",

"telefone": "5511987654321",

"dataNascimento": "1990-05-15",

"dataCadastro": "2026-03-16T09:30:00Z" }

Payload (Saída - Erro 404 Not Found):

{

"status": 404,

"erro": "Not Found",

"mensagem": "Usuário com ID 1 não encontrado." }

2.4. Listagem de Quadras

Retorna uma lista de todas as quadras disponíveis no sistema.

- Recurso: Quadras
- Endpoint: /api/quadras
- Verbo HTTP: GET
- Ação: Lista todas as quadras cadastradas, podendo aceitar parâmetros de filtro (ex: /api/quadras?tipoEsporte=Futsal).

Payload (Entrada): N/A (filtros via query parameters)

Payload (Saída - Sucesso 200 OK):

[{ "id": 101,

"nome": "Quadra Poliesportiva A",

"tipoEsporte": "Futsal",

"localizacao": "Rua das Flores, 123",

"capacidade": 10,

"valorHora": 80.00,

"disponivel": true }, { "id": 102,

"nome": "Quadra de Tênis B",

"tipoEsporte": "Tênis",

"localizacao": "Avenida Principal, 456",

"capacidade": 4,

"valorHora": 120.00,

"disponivel": true }]

Payload (Saída - Erro 500 Internal Server Error):

{

"status": 500,

"erro": "Internal Server Error",

"mensagem": "Erro ao listar quadras." }

2.5. Cadastro de Nova Quadra

Permite o registro de novas quadras no sistema.

- Recurso: Quadras
- Endpoint: /api/quadras
- Verbo HTTP: POST
- Ação: Cadastra uma nova quadra com suas características e disponibilidade.

Payload (Entrada):

{

"nome": "Quadra de Vôlei C",

"tipoEsporte": "Vôlei",

"localizacao": "Praça Central, S/N",

"capacidade": 12,

"valorHora": 90.00,

"disponivel": true }

Payload (Saída - Sucesso 201 Created):

{

"id": 103,

"nome": "Quadra de Vôlei C",

"tipoEsporte": "Vôlei",

"localizacao": "Praça Central, S/N",

"capacidade": 12,

"valorHora": 90.00,

"disponivel": true }

Payload (Saída - Erro 400 Bad Request):

{

"status": 400,

"erro": "Bad Request",

"mensagem": "Dados da quadra inválidos ou nome já existente." }

2.6. Consulta de Horários Disponíveis da Quadra

Permite verificar a disponibilidade de horários para uma quadra específica em uma determinada data.

- Recurso: Quadras
- Endpoint: /api/quadras/{id}/horarios
- Verbo HTTP: GET
- Ação: Retorna os horários disponíveis e reservados para uma quadra específica em uma data fornecida via query parameter (ex: ?data=2026-03-17).

Payload (Entrada): N/A (ID na URL, data via query parameter)

Payload (Saída - Sucesso 200 OK):

{

"quadraId": 101,

"data": "2026-03-17",

"horariosDisponiveis": [

"08:00-09:00",

"09:00-10:00",

"14:00-15:00",

"19:00-20:00"], "horariosReservados": [

"10:00-11:00",

"11:00-12:00"] }

Payload (Saída - Erro 404 Not Found):

{

"status": 404,

"erro": "Not Found",

"mensagem": "Quadra com ID 101 não encontrada." }

2.7. Criação de Reserva

Permite que um usuário crie uma nova reserva para uma quadra.

- Recurso: Reservas
- Endpoint: /api/reservas
- Verbo HTTP: POST
- Ação: Cria uma nova reserva de quadra para um usuário em um horário específico, verificando a disponibilidade.

Payload (Entrada):

{

"usuarioId": 1,

"quadraId": 101,

"dataReserva": "2026-03-17",

"horaInicio": "19:00",

"horaFim": "20:00",

"valorTotal": 80.00 }

Payload (Saída - Sucesso 201 Created):

{

"id": 501,

"usuarioId": 1,

"quadraId": 101,

"dataReserva": "2026-03-17",

"horaInicio": "19:00",

"horaFim": "20:00",

"valorTotal": 80.00,

"status": "PENDENTE_PAGAMENTO",

"dataCriacao": "2026-03-16T10:15:00Z" }

Payload (Saída - Erro 409 Conflict):

{

"status": 409,

"erro": "Conflict",

"mensagem": "Horário já reservado ou dados inválidos." }

2.8. Listagem de Reservas por Usuário

Retorna todas as reservas associadas a um usuário específico.

- Recurso: Reservas
- Endpoint: /api/reservas/usuario/{usuarioId}
- Verbo HTTP: GET
- Ação: Lista todas as reservas feitas por um usuário, identificadas pelo seu ID.

Payload (Entrada): N/A (ID do usuário na URL)

Payload (Saída - Sucesso 200 OK):

[{ "id": 501,

"usuarioId": 1,

"quadraId": 101,

"nomeQuadra": "Quadra Poliesportiva A",

"dataReserva": "2026-03-17",

"horaInicio": "19:00",

"horaFim": "20:00",

"valorTotal": 80.00,

"status": "PENDENTE_PAGAMENTO",

"dataCriacao": "2026-03-16T10:15:00Z" }, { "id": 502,

"usuarioId": 1,

"quadraId": 102,

"nomeQuadra": "Quadra de Tênis B",

"dataReserva": "2026-03-18",

"horaInicio": "10:00",

"horaFim": "11:00",

"valorTotal": 120.00,

"status": "CONFIRMADA",

"dataCriacao": "2026-03-15T15:00:00Z" }]

Payload (Saída - Erro 404 Not Found):

{

"status": 404,

"erro": "Not Found",

"mensagem": "Nenhuma reserva encontrada para o usuário com ID 1." }

2.9. Atualização de Status de Reserva

Permite a modificação do status de uma reserva existente.

- Recurso: Reservas
- Endpoint: /api/reservas/{id}/status
- Verbo HTTP: PATCH
- Ação: Atualiza o status de uma reserva específica (ex: para "CANCELADA", "CONFIRMADA", "PAGA").

Payload (Entrada):

{

"novoStatus": "CANCELADA" }

Payload (Saída - Sucesso 200 OK):

{

"id": 501,

"usuarioId": 1,

"quadraId": 101,

"dataReserva": "2026-03-17",

"horaInicio": "19:00",

"horaFim": "20:00",

"valorTotal": 80.00,

"status": "CANCELADA",

"dataCriacao": "2026-03-16T10:15:00Z",

"dataAtualizacao": "2026-03-16T11:00:00Z" }

Payload (Saída - Erro 400 Bad Request):

{

"status": 400,

"erro": "Bad Request",

"mensagem": "Status inválido ou reserva não pode ser atualizada." }

2.10. Processamento de Pagamento

Endpoint para processar o pagamento de uma reserva, alterando seu status para pago.

- Recurso: Pagamentos
- Endpoint: /api/pagamentos
- Verbo HTTP: POST
- Ação: Processa o pagamento de uma reserva específica, validando os dados e atualizando o status da reserva.

Payload (Entrada):

{

"reservaId": 501,

"metodoPagamento": "CARTAO_CREDITO",

"valorPago": 80.00,

"detalhesCartao": {

"numero": "**********1234",

"bandeira": "VISA",

"titular": "João Silva" } }

Payload (Saída - Sucesso 200 OK):

{

"id": 601,

"reservaId": 501,

"status": "APROVADO",

"valor": 80.00,

"metodoPagamento": "CARTAO_CREDITO",

"dataPagamento": "2026-03-16T11:30:00Z" }

Payload (Saída - Erro 400 Bad Request):

{

"status": 400,

"erro": "Bad Request",

"mensagem": "Falha no processamento do pagamento ou reserva já paga."

}